# CUESTIONARIO RESUELTO — UNIDAD 5: SUBSISTEMA DE ENTRADA/SALIDA (E/S)

> **Sistemas Operativos (UADE / Red Hat)** — Resolución pedagógica de las 19 preguntas de `Consignas y cuestionario (Unidad 5).pdf`, basada en el apunte `Unidad 5.pdf` (Tanenbaum cap. 5, Silberschatz cap. 13).

## Índice por módulos temáticos

| Módulo | Tema | Preguntas |
|---|---|---|
| **A** | Hardware de E/S: dispositivos y controladores | 1 – 4 |
| **B** | Direccionamiento de controladores y DMA | 5 – 6 |
| **C** | Objetivos del software de E/S | 7 – 10, 13 |
| **D** | Técnicas de E/S (programada, interrupciones, DMA) | 11 – 12 |
| **E** | Capas del software de E/S | 14 – 19 |

> 📝 **Nota sobre la pregunta 7:** el enunciado dice *"independencia de depósitos"*. En el apunte de cátedra el concepto se llama **independencia de dispositivos**; se responde con ese término (casi seguro es una errata de transcripción).

## Mapa mental rápido (para repasar antes del examen)

```
text
Programa de usuario  ──►  Biblioteca (printf, write) / Spooling (demonio)     ← Capa 4: espacio de usuario
                          Software independiente del dispositivo (VFS)         ← Capa 3: interfaz uniforme, búfer, errores, asignación
                          Drivers de dispositivos                              ← Capa 2: conocen los registros del controlador
                          Manejadores de interrupciones                        ← Capa 1: ocultan las interrupciones
──────────────────────────────────────────────────────────────────────────────
Hardware:  CPU ◄─(registros, comandos)─► Controlador (HW) ◄─(señales eléctricas)─► Dispositivo
```

# MÓDULO A — Hardware de E/S: dispositivos y controladores

### Pregunta 1: ¿Qué diferencias existen entre los dispositivos de E/S de bloque y los dispositivos de E/S de carácter?

> 💡 **En palabras simples (Para entenderlo de cero):** Un dispositivo de bloque es como un **fichero de cajones numerados**: podés abrir cualquier cajón en el orden que quieras. Uno de carácter es como una **canilla**: sale un chorro continuo y lo tomás en el orden en que viene, sin poder "volver atrás" a buscar una gota.

**Explicación y Concepto de Cátedra:**

| Característica | Dispositivo de bloque | Dispositivo de carácter |
|---|---|---|
| Cómo guarda/maneja la información | En **bloques de tamaño fijo** (de 512 B a 32 768 B), cada uno con su propia **dirección** | Envía o acepta un **flujo de caracteres** (bytes), sin estructura de bloque |
| Direccionable | **Sí** (se puede acceder a cualquier bloque) | **No** |
| Operación de búsqueda (*seek*) | **Sí** | **No tiene** |
| Transferencias | En unidades de uno o más **bloques completos** | Byte a byte / secuencial |
| Propiedad esencial | Cada bloque se lee/escribe **independientemente** de los demás | Los datos se consumen en el orden de llegada |
| Ejemplos | Discos duros, SSD, CD-ROM, memorias USB | Impresoras, teclados, mouse, interfaces de red (según el apunte) |

Observaciones de cátedra:

- La clasificación **no es perfecta**: los *relojes* no son de bloque ni de carácter (solo generan interrupciones a intervalos regulares) y las pantallas mapeadas en memoria tampoco encajan.
- Aun así es útil, porque el **sistema de archivos** trabaja con dispositivos de bloque *abstractos* y deja lo dependiente del hardware a los niveles inferiores.
- Linux agrega una tercera categoría: **dispositivos de red**, que se manejan con *sockets* y no con `read()/write()` sobre un archivo de `/dev`.

**📌 Ejemplo Práctico / En Linux:**

```
bash
ls -l /dev/sda /dev/tty0 /dev/null
# brw-rw---- 1 root disk 8, 0 ... /dev/sda    <- la 'b' inicial = dispositivo de
BLOQUE
# crw--w---- 1 root tty  4, 0 ... /dev/tty0   <- la 'c' inicial = dispositivo de
CARÁCTER
# crw-rw-rw- 1 root root 1, 3 ... /dev/null   <- 'c': carácter

lsblk          # lista los dispositivos de bloque (discos y particiones)
```

## Pregunta 2: Describa el componente de hardware denominado controlador de dispositivo.

> 💡 **En palabras simples (Para entenderlo de cero):** Es el **"traductor" electrónico** que está pegado al dispositivo. La CPU le da órdenes simples ("leé el bloque 500") y él se encarga de todo el trabajo sucio y mecánico/eléctrico. Es como el **mozo de un restaurante**: vos pedís "una milanesa" y él se ocupa de hablar con la cocina.

**Explicación y Concepto de Cátedra:** Una unidad de E/S suele tener **dos partes**:

1. **Componente mecánico:** el dispositivo en sí (plato del disco, cabezal, motor, celdas flash).
2. **Componente electrónico:** el **controlador de dispositivo (HW)** o **adaptador**. Puede ser un chip en la placa madre o una tarjeta que se inserta en una ranura de expansión.

Características:

- Tiene **registros** (control, estado, datos) para comunicarse con la CPU y a menudo un **búfer de datos** interno.
- Un mismo controlador puede manejar varios dispositivos idénticos (2, 4 u 8).
- Si la interfaz controlador–dispositivo es **estándar** (IDE, SATA, SCSI, USB, FireWire, NVMe), cualquier fabricante puede construir dispositivos compatibles.
- **Su trabajo típico (ejemplo del disco):** el disco entrega un flujo **serial de bits** (preámbulo + datos del sector + ECC). El controlador lo convierte en un **bloque de bytes**, lo arma en su búfer interno, **verifica la suma de comprobación (ECC/checksum)**, corrige errores y recién entonces el bloque puede copiarse a la memoria principal.
- Ventaja: el sistema operativo solo **inicializa el controlador con unos pocos parámetros** y éste maneja los detalles (ej.: el controlador de un monitor CRT maneja el haz de electrones).

📌 **Ejemplo Práctico / En Linux:**

```
bash
lspci | grep -i -E "sata|nvme|ethernet|vga"   # controladores (HW) en el bus PCI
# 00:17.0 SATA controller: Intel Corporation ...
lspci -k                                      # muestra además el driver (SW) que lo
maneja
```

### Pregunta 3: Describa la E/S con asignación de puertos. ¿Por qué motivo la CPU no puede manipular datos almacenados en un medio secundario (HDD/SSD) de manera directa? ¿Cuál es el papel del controlador de dispositivo (HW) en este asunto?

> 💡 **En palabras simples (Para entenderlo de cero):** La E/S por puertos es como tener **un teléfono interno con números de interno propios** (puerto 0x60, 0x3F8...) que solo sirven para hablar con los dispositivos, separados de las "direcciones de la memoria". Y la CPU no puede leer un disco directamente porque el disco es **lento y mecánico/complejo**: habla otro idioma (señales eléctricas, sectores, ECC). Necesita un **intérprete**: el controlador.

**Explicación y Concepto de Cátedra:**

**a) E/S con asignación de puertos (*Port-Mapped I/O*):**

- A **cada registro de control** del controlador se le asigna un **número de puerto de E/S** (entero de 8 o 16 bits).
- El conjunto de todos los puertos forma el **espacio de puertos de E/S**, **separado** del espacio de direcciones de memoria y **protegido**: solo el sistema operativo (modo kernel) puede usarlo.
- La CPU lo accede con **instrucciones especiales**:
  - `IN REG, PUERTO` → lee el registro de control `PUERTO` y lo guarda en el registro de CPU `REG`.
  - `OUT PUERTO, REG` → escribe el contenido de `REG` en el registro de control.
- Una línea de control del bus indica si el acceso es a **memoria** o a **E/S**.
- **Desventaja:** `IN` / `OUT` no existen en C/C++ → hace falta **código ensamblador**, lo que agrega sobrecarga. Se usó en mainframes (IBM 360) y es soportado por x86.

**b) ¿Por qué la CPU no puede manipular directamente datos del HDD/SSD?**

1. La CPU solo ejecuta instrucciones sobre **registros y memoria principal (RAM)**; el medio secundario **no está en su espacio de direcciones**.
2. El disco entrega los datos como **flujo serial de bits** con preámbulo, sincronización y ECC, a un ritmo **fijo** que la CPU no controla, y con tiempos mecánicos (o de celdas flash) **órdenes de magnitud más lentos** que la CPU.
3. Los comandos reales del disco (mover cabezal, sectores, cilindros, programar celdas) son de **muy bajo nivel eléctrico**.

**c) Papel del controlador (HW):**

- Es el **intermediario**: del lado de la CPU expone una interfaz de **alto nivel** (registros, direcciones, comandos en bloque); del lado del dispositivo maneja **señales eléctricas, temporización y**

**protocolos** de bajo nivel.

- Arma el bloque en su **búfer interno**, verifica el checksum/ECC y, solo si es correcto, el bloque pasa a la RAM (por E/S programada o DMA). Recién ahí la CPU puede trabajar con los datos.

📌 **Ejemplo Práctico / En Linux:**

```
bash
sudo cat /proc/ioports | head -20      # rangos de PUERTOS de E/S asignados (ej.:
0060-0060 : keyboard)
sudo cat /proc/iomem  | head -20       # rangos de E/S mapeada en MEMORIA
```

### Pregunta 4: Explique por qué la CPU interactúa con un controlador de dispositivo a través de código de programación, mientras que el controlador de dispositivo interactúa con un periférico mediante señales y protocolos electrónicos.

> 💡 **En palabras simples (Para entenderlo de cero):** Es una **cadena de traductores**: vos le das una orden escrita a un gerente (CPU → controlador, con "programas"); el gerente no mueve una pala: le grita instrucciones eléctricas al operario (controlador → periférico, con "señales"). Cada tramo habla el idioma que le conviene.

**Explicación y Concepto de Cátedra:**

| Interfaz | Nivel | Naturaleza | Cómo se expresa |
|---|---|---|---|
| **CPU ⇄ Controlador** | Alto nivel (registros, direcciones, comandos en bloque) | **Lógica y programable** | **Código**: drivers escritos en **C/C++**, históricamente **assembly** para rutinas críticas y hoy también **Rust** en el kernel Linux |
| **Controlador ⇄ Periférico** | Muy bajo nivel (pulsos, estados de líneas, sectores magnéticos) | **Física y electrónica** | **Especificaciones y protocolos de hardware** (SATA, NVMe, USB, PCIe, I²C, SPI) con **pulsos de voltaje, patrones de reloj (*clock*) y secuencias de bits** para mover cabezales, activar celdas flash o leer sensores |

Razones:

1. **La CPU es programable**: su mundo son instrucciones y datos; por eso puede ser dirigida por **software** (el driver escribe en registros del controlador).
2. **El periférico es físico**: un motor, un láser o una celda flash solo responden a **tensiones, tiempos y señales**, no a instrucciones de programa.

3. El **controlador** existe justamente para **traducir** ambos mundos y así liberar al sistema operativo de programar, por ejemplo, la exploración analógica de un tubo CRT.

📌 **Ejemplo Práctico / En Linux:**

```
bash
lsmod | head                 # módulos (código de drivers) cargados en el kernel
modinfo ahci                 # driver del controlador SATA (código, lado
CPU↔controlador)
sudo dmesg | grep -i -E "ata[0-9]|nvme"   # el kernel negociando el protocolo con el
dispositivo
```

# MÓDULO B — Direccionamiento de controladores y DMA

### Pregunta 5: Describa la E/S con asignación de memoria.

> 💡 **En palabras simples (Para entenderlo de cero):** En vez de tener un "teléfono aparte" para los dispositivos (puertos), **le reservamos algunas direcciones de la memoria** y las usamos como si fueran botones del dispositivo. Escribir en esa dirección = "apretar el botón". Para el programador, hablar con el hardware es **como escribir en una variable**.

**Explicación y Concepto de Cátedra:**

- *Memory-Mapped I/O (MMIO)*: **todos los registros de control** (y a veces los búferes de datos) se asignan al **espacio de direcciones de memoria**. A cada registro se le da una **dirección única a la que no hay RAM asignada**; normalmente en la parte alta del espacio de direcciones.
- La CPU lee/escribe con las instrucciones normales de memoria ( `MOV` , `TEST` , etc.).
- **Esquema híbrido** (el de los Pentium/PC): búferes de datos mapeados en memoria (ej. 640K–1M) + puertos de E/S separados (0 a 64K) para los registros de control. **Linux usa un esquema híbrido**: mayoritariamente MMIO para hardware de alto rendimiento y también soporta puertos en x86.

**Ventajas:**

1. El driver puede escribirse **100 % en C** (los registros son como variables; no se necesita ensamblador ni `IN` / `OUT` ).

2. **No requiere protección especial:** basta con no mapear esas direcciones en el espacio virtual de los usuarios. Si cada dispositivo está en una página distinta, el SO puede **dar a un usuario control de un dispositivo y no de otros** modificando la tabla de páginas.
3. **Cualquier instrucción** que referencie memoria sirve para los registros de control (ej.: `TEST` para ver si un registro vale 0).

**Desventajas:**

1. Hay que **deshabilitar la caché selectivamente** (por página) para esas direcciones, o se leerían valores viejos → más complejidad de hardware y SO.
2. Con un único espacio de direcciones, **todos los módulos de memoria y dispositivos deben examinar cada referencia**. Con **buses duales** (bus de memoria de alta velocidad + bus de E/S) los dispositivos no ven las direcciones del bus de memoria → soluciones: enviar primero a memoria y si no responde probar otros buses, un dispositivo "husmeador", o **filtrar en el puente PCI** con registros de rango cargados al inicio (el *Northbridge*, hoy integrado en la CPU como **IMC**).

📌 **Ejemplo Práctico / En Linux:**

```
bash
sudo cat /proc/iomem | grep -i -E "pci|nvme|video"   # regiones de memoria usadas
por dispositivos
```

```
c
/* Idea de un driver con MMIO (pseudocódigo del kernel): */
volatile unsigned int *reg_estado = (unsigned int *)0xFEDC0000; /* dirección mapeada
*/
while ((*reg_estado & LISTO) == 0) ;   /* leer un registro = leer una variable */
```

## Pregunta 6: Describa el acceso directo a la memoria (DMA).

> 💡 **En palabras simples (Para entenderlo de cero):** Imaginá que tenés que mover 500 cajas del camión al depósito. Sin DMA, **vos (la CPU) cargás cada caja**. Con DMA, le **contratás un montacargas**: le decís "mové 500 cajas desde acá hasta allá" y vos seguís trabajando; cuando termina, **te avisa con un timbre** (interrupción).

**Explicación y Concepto de Cátedra:**

- **DMA (*Direct Memory Access*)** permite que un **controlador de DMA** transfiera datos entre un dispositivo y la memoria principal **sin que la CPU copie cada byte**. Evita malgastar tiempo de

CPU.

- Requiere un **controlador de DMA** con acceso al bus **independiente de la CPU**. Puede estar integrado en el controlador del dispositivo (uno por dispositivo, *bus mastering*), ser uno solo en la placa que atienda varios dispositivos, o, en CPUs modernas, estar dentro del **IMC** (controlador de memoria integrado) conectado a las líneas PCIe.
- Tiene **registros**: de **dirección de memoria**, **contador de bytes** y uno o más de **control** (puerto de E/S, sentido de la transferencia, unidad de transferencia, nº de bytes por ráfaga).

**Pasos de una lectura de disco con DMA:**

1. La **CPU programa** el controlador de DMA (dirección destino, cantidad de bytes) y le ordena al **controlador de disco** que lea el bloque a su búfer interno y verifique el checksum.
2. El controlador de DMA envía una **petición de lectura** al controlador de disco por el bus.
3. El disco escribe la palabra en la **memoria** (ciclo de bus estándar).
4. El disco envía un **reconocimiento (ACK)**. El DMA **incrementa la dirección y decrementa el contador**. Si el contador > 0, repite 2–4.
5. Cuando el contador llega a 0, el DMA **interrumpe a la CPU**: la transferencia está completa. El bloque **ya está en memoria**.

**Modos de operación:**

- **Una palabra a la vez (robo de ciclo, *cycle stealing*):** el DMA "roba" un ciclo de bus a la CPU de vez en cuando, retrasándola levemente.
- **Modo bloque / ráfaga (*burst*):** adquiere el bus, hace una serie de transferencias y lo libera. Es más eficiente, pero puede bloquear a la CPU y a otros dispositivos un tiempo considerable.

**Otros puntos:** la mayoría de los DMA usan **direcciones físicas** (el SO debe traducir la dirección virtual del búfer); algunos usan virtuales y la MMU traduce. El disco usa un **búfer interno** para verificar el checksum antes de transferir y porque los bits llegan a velocidad constante aunque el bus esté ocupado. **Cuándo no conviene:** si la CPU es más rápida que el DMA y no tiene otra cosa que hacer (o en sistemas embebidos baratos).

📌 **Ejemplo Práctico / En Linux:**

```
bash
cat /proc/dma                                # canales DMA (ISA) registrados
sudo dmesg | grep -i dma                     # mensajes de DMA del kernel
dd if=/dev/zero of=/tmp/prueba bs=1M count=500 oflag=direct   # 'direct' evita la
caché de páginas
```

# MÓDULO C — Objetivos del software de E/S

### Pregunta 7: Describa el concepto de independencia de dispositivos (enunciado: "independencia de depósitos").

> 💡 **En palabras simples (Para entenderlo de cero):** Es como un **enchufe universal**: tu lámpara funciona igual sin importar si la pared es de la casa A o de la casa B. Un programa debería **leer un archivo igual** sin importar si está en un disco, un pendrive o un DVD.

**Explicación y Concepto de Cátedra:**

- Es un **objetivo clave del software de E/S**: debe ser posible escribir programas que accedan a **cualquier dispositivo sin especificarlo por adelantado** ni modificarse para cada uno.
- Ejemplo: un programa que lee un archivo funciona con el disco duro, CD-ROM, DVD o USB sin cambios. El comando `sort <entrada >salida` funciona con datos de cualquier disco o teclado, y manda la salida a cualquier disco o pantalla.
- **Quien lo logra es el SO**, que oculta que los dispositivos son distintos y requieren secuencias de comandos muy diferentes.
- Va de la mano con la **denominación uniforme**: el nombre de un archivo o dispositivo es una cadena o entero independiente del dispositivo. En UNIX, todos los discos se integran en **una única jerarquía** y todo se direcciona por **ruta** (ej.: montar un USB en `/usr/ast/respaldo` y copiar a `/usr/ast/respaldo/lunes` lo escribe en el USB).
- Se implementa con la **interfaz uniforme de drivers** y el **software independiente del dispositivo** (Preguntas 14 y 18).

📌 **Ejemplo Práctico / En Linux:**

```
bash
sort < datos.txt > ordenado.txt      # entrada y salida: archivos
sort < /dev/stdin > /dev/null        # el mismo comando con teclado y dispositivo
nulo
cp archivo.txt /mnt/usb/             # igual comando para disco interno o pendrive
echo "hola" > /dev/tty               # 'todo es un archivo': la terminal también
```

## Pregunta 8: ¿Qué diferencias existen entre las transferencias sincrónicas y asincrónicas?

> 💡 **En palabras simples (Para entenderlo de cero): Sincrónica:** pedís una pizza y **te quedás parado en la puerta esperando** hasta que llegue. **Asincrónica:** pedís la pizza, **seguís con tus cosas** y cuando llega **suena el timbre**.

**Explicación y Concepto de Cátedra:**

| | **Sincrónica (de bloqueo)** | **Asincrónica (controlada por interrupciones)** |
|---|---|---|
| ¿Qué hace la CPU/proceso? | **Inicia** la operación y **espera** a que termine antes de continuar | **Inicia** la transferencia y se va a **hacer otra cosa** |
| Cómo se entera del fin | Cuando vuelve de la llamada bloqueante | Llega una **interrupción** |
| Facilidad de programación | **Mucho más fácil** (el `read` suspende al proceso hasta que haya datos) | Más compleja |
| Aprovechamiento de la CPU | Peor (o la usa otro proceso si el SO planifica) | Mejor |

- **La mayoría de las operaciones físicas de E/S son asíncronas.** Pero los programas de usuario son más fáciles de escribir si la E/S es de bloqueo, así que **es tarea del SO hacer que operaciones realmente controladas por interrupciones parezcan de bloqueo** para los programas de usuario (el proceso se bloquea y otro corre mientras tanto).

📌 **Ejemplo Práctico / En Linux:**

```
c
/* Sincrónica: read() bloquea al proceso hasta que haya datos */
n = read(fd, buf, 100);

/* Asincrónica: O_NONBLOCK devuelve de inmediato (-1, errno=EAGAIN) si no hay datos
*/
fd = open("/dev/ttyS0", O_RDONLY | O_NONBLOCK);
```

```
bash
strace -T cat /etc/passwd 2>&1 | grep read   # -T muestra cuánto bloqueó cada read
```

## Pregunta 9: ¿Es correcto afirmar que el DMA es un mecanismo de E/S asíncrono? Justifique su respuesta.

> 💡 **En palabras simples (Para entenderlo de cero):** Sí. Es el **montacargas del ejemplo**: la CPU lo pone a trabajar y **no se queda esperando**; sigue con otras tareas hasta que el DMA la **interrumpe** al terminar.

**Explicación y Concepto de Cátedra:**

**VERDADERO.**

Justificación:

1. Con DMA, la CPU solo **programa** el controlador de DMA (dirección, contador, control) y **continúa ejecutando otro trabajo**; la transferencia ocurre **en paralelo** por el bus, sin intervención constante de la CPU.
2. La **finalización se notifica con una interrupción**, característica de la E/S asíncrona (la CPU no hace espera activa/sondeo).
3. Además reduce las interrupciones: **una por búfer** en vez de **una por carácter**, y el DMA "se parece a la E/S programada pero hecho por el controlador de DMA en lugar de la CPU".

⚠️ **Matiz para la defensa oral:** es asíncrono *respecto de la CPU*. El **proceso de usuario** que pidió la operación puede igualmente estar **bloqueado** (E/S sincrónica a nivel de llamada al sistema), mientras la CPU ejecuta otros procesos. Además, el DMA **compite con la CPU por el bus** (robo de ciclo o ráfaga).

📌 **Ejemplo Práctico / En Linux:**

```
bash
cat /proc/interrupts | grep -i -E "nvme|ahci|eth"   # una interrupción por
transferencia completa (no por byte)
```

## Pregunta 10: ¿Qué diferencias existen entre los dispositivos de E/S compartidos y los dedicados?

> 💡 **En palabras simples (Para entenderlo de cero): Compartido** = una **biblioteca**: muchos pueden leer a la vez. **Dedicado** = un **baño con llave**: entra uno solo y hasta que no sale, nadie más usa.

**Explicación y Concepto de Cátedra:**

| | Compartidos | Dedicados |
|---|---|---|
| Uso | Muchos usuarios/procesos **a la vez** | **Un solo usuario/proceso** hasta que termine |
| Ejemplos | **Discos** | Unidades de **cinta**, **escáner**, grabadores de CD-ROM, impresoras (acceso directo) |
| Problemas | Se resuelve con el sistema de archivos | Si dos usuarios mezclan bloques en una cinta, **no funciona**; además introducen **interbloqueos (deadlocks)** |

- El **SO debe manejar ambos** sin generar problemas.
- **Asignación y liberación de dispositivos dedicados:** el SO acepta o rechaza peticiones según disponibilidad. Opciones: (a) hacer `open` del archivo especial: si está ocupado, **falla**; `close` lo libera. (b) Mecanismos especiales de solicitar/liberar: si no está disponible, el proceso **se bloquea en una cola**.
- Técnica para volver "compartible" un dispositivo dedicado: **spooling** (cola con demonio), Pregunta 19.

📌 **Ejemplo Práctico / En Linux:**

```
bash
lsblk                                   # disco: compartido por muchos procesos
sudo fuser -v /dev/sr0                  # ¿quién usa la lectora/grabadora (dedicada)?
lpstat -t                               # cola de impresión (spooling)
```

## Pregunta 13: ¿Es correcto afirmar que los errores de E/S deben ser manejados lo más lejos posible del hardware? Justifique la respuesta.

> 💡 **En palabras simples (Para entenderlo de cero):** Es al revés. Si se te cae un vaso, lo **levantás vos mismo** en vez de llamar al gerente. Los errores se arreglan **lo más cerca posible de donde ocurren**, y solo si no se puede, se avisa "hacia arriba".

**Explicación y Concepto de Cátedra:**

**FALSO.** Los errores de E/S deben manejarse **lo más cerca del hardware que sea posible**.

Justificación (apunte de cátedra):

1. Si el **controlador** detecta un error de lectura, debe tratar de **corregirlo por sí mismo** (ECC). Si no puede, lo maneja el **driver** (ej.: **reintentando** leer el bloque).

2. Muchos errores son **transitorios** (polvo en la cabeza lectora) y **desaparecen al repetir** la operación.
3. **Solo si los niveles inferiores no pueden resolverlo**, los niveles superiores deben enterarse. Así la recuperación es **transparente** a los niveles superiores.
4. **Clasificación de errores:**
    - *De programación* (escribir en un teclado, buffer inválido, dispositivo inexistente): se devuelve un **código de error** al que llamó.
    - *De E/S reales* (bloque dañado, cámara apagada): decide el **driver**; si no sabe, lo pasa al software independiente del dispositivo (reintentar, ignorar, matar el proceso, mostrar diálogo o fallar con código de error).
    - *Críticos* (directorio raíz o lista de bloques libres destruidos): mostrar mensaje y terminar.

- El **marco de trabajo** para reportar errores es **independiente del dispositivo**, pero los errores específicos los trata el driver.

📌 **Ejemplo Práctico / En Linux:**

```
bash
sudo dmesg | grep -i -E "I/O error|ata[0-9].*error|retry"   # reintentos del driver
a nivel kernel
sudo smartctl -a /dev/sda                                   # errores corregidos por
el propio disco
ls /archivo_inexistente; echo $?                            # error de programación
-> código (2, ENOENT)
```

# MÓDULO D — Técnicas de E/S (programada, interrupciones, DMA)

### Pregunta 11: ¿Qué es la E/S programada? Mencione sus principales ventajas y debilidades.

> 💡 **En palabras simples (Para entenderlo de cero):** Es como **mirar el microondas cada 2 segundos** preguntándote "¿ya está?" sin hacer otra cosa. La CPU se queda **preguntando una y otra vez** al dispositivo si está listo.

**Explicación y Concepto de Cátedra:**

- **E/S programada:** la forma más simple de E/S; **la CPU hace todo el trabajo**.
- **Funcionamiento** (ejemplo: imprimir "ABCDEFGH"):
    1. El proceso arma la cadena en un búfer de usuario y abre la impresora ( `open` ).
    2. Hace la llamada al sistema para imprimir; el SO **copia el búfer al espacio del kernel**.
    3. El SO comprueba si la impresora está lista; cuando lo está, **copia un carácter** a su registro de datos.
    4. **Sondea** el registro de estado hasta que la impresora vuelva a estar lista y repite hasta terminar. Luego devuelve el control al proceso.
- Ese ciclo continuo se llama **sondeo (*polling*), ocupado en espera o espera activa (*busy waiting*)**.

| ✅ Ventajas | ❌ Debilidades |
|---|---|
| **Muy simple** de implementar | **Ocupa la CPU al 100 %** hasta terminar la E/S (desperdicio) |
| No requiere hardware especial ni manejo de interrupciones | Ineficiente si la CPU tiene otras tareas y el dispositivo es lento |
| **Razonable** si el dispositivo responde muy rápido (ej.: la impresora solo copia a un búfer interno) | |
| **Adecuada en sistemas embebidos** donde la CPU no tiene otra cosa que hacer | |

📌 **Ejemplo Práctico / En Linux:**

```
c
copy_from_user(p, buffer, count);              /* 1. copiar datos al kernel */
for (i = 0; i < count; i++) {
    while (*printer_status_reg != READY) ;     /* 2. SONDEO: espera activa */
    *printer_data_register = p[i];             /* 3. un carácter por vez */
}
```

```
bash
yes > /dev/null &  top      # un proceso en espera activa consume ~100 % de un
núcleo
```

## Pregunta 12: ¿Qué es la E/S controlada por interrupciones? Mencione sus principales ventajas y debilidades.

> 💡 **En palabras simples (Para entenderlo de cero):** En vez de mirar el microondas, **te ponés a hacer otra cosa y esperás a que suene el "ding"**. El dispositivo **avisa** (interrupción) cuando termina.

**Explicación y Concepto de Cátedra:**

- **Método** en el que el dispositivo **avisa a la CPU generando una interrupción** cuando está listo o terminó. La CPU **no espera activamente**.
- **Ejemplo:** una impresora que imprime 100 car/seg tarda 10 ms por carácter; ese tiempo alcanza para un **cambio de contexto** y correr otro proceso.
- **Funcionamiento:**
    1. En la llamada al sistema, el búfer se copia al kernel y se envía el **primer carácter**.
    2. La CPU **llama al planificador** y ejecuta **otro proceso**; el que pidió imprimir queda **bloqueado**.
    3. Cuando la impresora termina un carácter, genera una **interrupción**: se guarda el estado del proceso actual y se ejecuta el **manejador de interrupciones** de la impresora.
    4. Si no hay más caracteres, **desbloquea** al proceso de usuario; si los hay, imprime el siguiente, **reconoce la interrupción** y regresa al proceso interrumpido.

| ✅ Ventajas | ❌ Debilidades |
|---|---|
| **La CPU queda libre** para otros procesos mientras espera (**no hay espera activa**) | Ocurre **una interrupción por cada carácter** |
| Mucho mejor aprovechamiento en sistemas multiprogramados | Las interrupciones **toman tiempo** (guardar registros, contexto, MMU/TLB…): **sobrecarga (*overhead*)** |
| | Se desperdicia algo de tiempo de CPU; la solución es el **DMA** |

**📌 Ejemplo Práctico / En Linux:**

```
bash
cat /proc/interrupts                # contador de interrupciones por dispositivo y CPU
watch -n1 "grep -E 'eth|enp|nvme' /proc/interrupts"   # ver crecer los contadores al usar red/disco
```

# MÓDULO E — Capas del software de E/S

### Pregunta 14: Enumere las principales capas del software de E/S.

> 💡 **En palabras simples (Para entenderlo de cero):** Como un **edificio de 4 pisos**: arriba está el usuario y abajo el hardware. Cada piso solo habla con el de arriba y el de abajo.

**Explicación y Concepto de Cátedra:** El software de E/S se organiza en **cuatro capas** (de abajo hacia arriba), cada una con una función bien definida y una interfaz bien definida con las capas adyacentes:

| N.º | Capa | Función principal |
|---|---|---|
| 1 | **Manejadores de interrupciones** | Ocultan las interrupciones en las "entrañas" del SO |
| 2 | **Drivers (controladores de software) de dispositivos** | Código específico de cada dispositivo; hablan con los registros del controlador |
| 3 | **Software de E/S independiente del dispositivo** | Funciones comunes: interfaz uniforme, búfer, errores, asignación de dispositivos dedicados, tamaño de bloque uniforme |
| 4 | **Software de E/S en espacio de usuario** | Bibliotecas ( `printf` , `write` …) y *spooling* (demonios) |

*(Por debajo de todas está el hardware: controlador y dispositivo.)* Las capas 1–3 residen en el **kernel**; la 4 está **fuera**.

📌 **Ejemplo Práctico / En Linux:** Al ejecutar `printf("hola")` : capa 4 ( `printf` de glibc → `write` ) → capa 3 (VFS) → capa 2 (driver de la terminal/consola) → capa 1 (manejador de interrupción cuando el hardware termina).

```
bash
strace -e trace=write echo hola    # se ve el write() que cruza de la capa 4 a las
capas del kernel
```

## Pregunta 15: Describa la capa del software de E/S destinada a manejar interrupciones.

> 💡 **En palabras simples (Para entenderlo de cero):** Es el **portero que atiende el timbre**: cuando suena (interrupción) hace lo urgente y **despierta al empleado (driver)** que estaba dormido esperando. Así el resto de la casa ni se entera del timbre.

**Explicación y Concepto de Cátedra:**

- Las interrupciones **no se pueden evitar**; deben **ocultarse en las profundidades del SO** para que el resto sepa lo menos posible de ellas.
- **Mejor manera:** el driver que inicia una operación de E/S **se bloquea** (con `down` en un semáforo, `wait` en una variable de condición o `receive` de un mensaje) hasta que ocurra la interrupción. El manejador hace lo necesario y **lo desbloquea** ( `up` , `signal` o enviando un mensaje). Funciona mejor si los drivers son procesos del kernel con su propio estado, pila y contador de programa.
- **Pasos (varían según la máquina):**
    1. Guardar los registros (incluido el **PSW**) no guardados por el hardware.
    2. Establecer el contexto (TLB, MMU, tabla de páginas).
    3. Establecer una pila.
    4. Reconocer (ACK) al controlador de interrupciones / rehabilitar interrupciones.
    5. Copiar los registros a la tabla de procesos.
    6. Ejecutar el **procedimiento de servicio** (lee información del controlador que interrumpió).
    7. Elegir qué proceso ejecutar a continuación (el desbloqueado puede tener mayor prioridad).
    8. Establecer el contexto de la MMU para ese proceso.
    9. Cargar los registros del nuevo proceso, incluyendo su PSW.
    10. Ejecutar el nuevo proceso.
- Procesar una interrupción **consume muchas instrucciones** de CPU, sobre todo con memoria virtual.
- En Linux moderno: **parte alta (*top half*)** hace lo mínimo e inmediato; **parte baja (*bottom half*)** hace el trabajo pesado más tarde, siempre dentro del kernel.
- **Interrupciones precisas** (PC guardado, todo lo anterior ejecutado, nada posterior ejecutado, estado conocido) **vs. imprecisas** (no cumplen y complican al SO).

📌 **Ejemplo Práctico / En Linux:**

```
bash
cat /proc/interrupts          # IRQ, contadores por CPU, dispositivo que la origina
cat /proc/softirqs            # trabajo diferido (bottom halves)
```

## Pregunta 16: Describa el software de E/S conocido como driver.

> 💡 **En palabras simples (Para entenderlo de cero):** Es el **manual de instrucciones convertido en programa** para un dispositivo específico. Un mouse y un disco se manejan de forma totalmente distinta, así que cada uno necesita su **"conductor" especializado**.

**Explicación y Concepto de Cátedra:**

- Cada controlador (HW) tiene registros de comando y de estado cuya cantidad y significado **varían radicalmente** entre dispositivos (mouse vs. disco). Por eso cada dispositivo necesita **código específico: el driver**, normalmente **escrito por el fabricante** para varios sistemas operativos.
- Cada driver maneja **un tipo** de dispositivo o una **clase** de dispositivos estrechamente relacionados (un driver SATA sirve para varios discos SATA).
- **Categorías:** drivers de **bloque**, de **carácter** y, en Linux, de **red**.
- Todos aceptan una **interfaz estándar** (procedimientos para leer un bloque o escribir una cadena de caracteres).
- **Carga:** antes el kernel se **recompilaba** para agregar un driver; hoy se **cargan dinámicamente** (módulos).
- **Funciones del driver:**
    1. Aceptar peticiones abstractas de lectura/escritura del software independiente del dispositivo.
    2. **Validar parámetros** y traducir (ej.: bloque lineal → cabeza/pista/sector/cilindro).
    3. **Inicializar** el dispositivo y gestionar la energía y registros de eventos.
    4. Si está ocupado, **encolar** la petición.
    5. **Emitir comandos** escribiendo en los registros del controlador.
    6. **Bloquearse** hasta la interrupción (o continuar si es inmediato).
    7. **Comprobar errores** y devolver datos y estado al nivel superior.
    8. Atender la siguiente petición en cola.
- **Debe ser reentrante** (pueden llegar interrupciones mientras corre) y tolerar **conexión en caliente** (***hot-plug***).

- El driver **no hace llamadas al sistema**, pero puede llamar a **procedimientos del kernel** (asignar páginas fijas, MMU, temporizadores, DMA, controlador de interrupciones).

📌 **Ejemplo Práctico / En Linux:**

```
bash
lsmod | head                # drivers (módulos) cargados
modinfo e1000e              # información de un driver de red
sudo modprobe -r usb_storage && sudo modprobe usb_storage   # descarga/carga
dinámica
ls /lib/modules/$(uname -r)/kernel/drivers
```

## Pregunta 17: ¿Es correcto afirmar que los drivers (SW) forman parte del sistema operativo? Argumente su respuesta.

> 💡 **En palabras simples (Para entenderlo de cero):** Sí, en general: aunque los **escribe el fabricante** del dispositivo, **corren dentro del SO** (en el kernel) porque necesitan tocar el hardware directamente. Es como un **repuesto de otra marca que se instala dentro del motor**.

**Explicación y Concepto de Cátedra:**

**VERDADERO** (con matices).

Argumentos:

1. Para **acceder a los registros** del controlador, el driver por lo general **tiene que formar parte del kernel** y ejecutarse en **modo kernel** (privilegiado). En un kernel **monolítico** (clásico en Linux) **todo el código del driver corre dentro del espacio del kernel**, dividido en *top half* y *bottom half*.
2. Se ubican **debajo del resto del SO** y siguen un **modelo y una interfaz estándar** definidos por el SO (los diseñadores no conocen el código de terceros que se instalará).
3. Un driver defectuoso puede **colgar o corromper** todo el sistema, lo que demuestra que opera con los privilegios del SO.
4. Se cargan **dinámicamente** como **módulos del kernel**.

**Matices:**

- Aunque los **escribe un tercero** (fabricante), se integran al kernel.
- **Excepciones:** es posible construir drivers en **espacio de usuario** (con syscalls para leer/escribir los registros); esto da más confiabilidad al aislar fallas (ej.: **MINIX 3**); en Linux existen **FUSE** (sistemas de archivos) y **UIO** (dispositivos).

- **Windows** usa arquitectura **híbrida**: una parte corre en el kernel y otra puede correr en espacio de usuario.

📌 **Ejemplo Práctico / En Linux:**

```
bash
lsmod                          # módulos del kernel = drivers integrados al SO en
ejecución
ls /lib/modules/$(uname -r)/kernel/drivers/   # drivers distribuidos con el kernel
mount | grep fuse              # ejemplo de "driver" (FS) en espacio de usuario
```

## Pregunta 18: ¿Cuál es la función básica del software independiente del dispositivo de E/S?

> 💡 **En palabras simples (Para entenderlo de cero):** Es la **recepción común** del edificio: hace lo que **todos los dispositivos necesitan** (búfer, errores, permisos) y le muestra al usuario **una única ventanilla** igual para todos.

**Explicación y Concepto de Cátedra: Función básica:** realizar las **funciones de E/S comunes a todos los dispositivos** y **proveer una interfaz uniforme** al software de nivel de usuario. El límite exacto con los drivers depende del sistema.

Funciones típicas:

1. **Interfaz uniforme para los drivers:** sin interfaz estándar, cada driver nuevo exigiría **modificar el SO** y mucho esfuerzo; con una interfaz común, agregar un dispositivo no cambia el SO. Incluye **denominación** (nombre → driver) y **protección**.
2. **Uso de búfer** (*buffering*): desacopla velocidades y tamaños (esquemas: sin búfer, búfer en espacio de usuario, búfer en kernel, **doble búfer**, **búfer circular**). Su desventaja: demasiadas copias bajan el rendimiento (ej.: un paquete de red se copia 5 veces).
3. **Reporte de errores** (marco independiente del dispositivo).
4. **Asignación y liberación de dispositivos dedicados.**
5. **Tamaño de bloque independiente del dispositivo:** oculta diferentes tamaños de sector tratando varios sectores como un **bloque lógico**; también unifica dispositivos de carácter que envían byte a byte vs. en unidades grandes.

📌 **Ejemplo Práctico / En Linux:**

```
bash
blockdev --getss --getbsz /dev/sda    # tamaño de sector físico vs. bloque lógico
free -h                               # 'buff/cache': búferes del kernel
cat /dev/zero | head -c 10 | xxd      # misma interfaz (read/write) para cualquier
"archivo"
```

## Pregunta 19: Describa la porción de software de E/S en el espacio de usuario.

> 💡 **En palabras simples (Para entenderlo de cero):** Es la parte "de **cara al cliente**" que vive fuera del kernel: las **herramientas de las bibliotecas** ( `printf` , `write` ) y los **empleados de fila** (demonios de spooling) que organizan quién usa el dispositivo y cuándo.

**Explicación y Concepto de Cátedra:** Aunque casi todo el software de E/S está en el kernel, una pequeña porción está **fuera del kernel** y se compone de:

1. **Procedimientos de biblioteca** vinculados a los programas de usuario:
    - Las **llamadas al sistema de E/S** se hacen mediante procedimientos de biblioteca (ej.: `cuenta = write(da, bufer, nbytes);` el *wrapper* `write` coloca los parámetros y dispara la syscall).
    - **Formateo de E/S:** `printf` toma una cadena de formato, construye una cadena ASCII y llama a `write` ; `scanf` lee y guarda en variables. La biblioteca estándar `stdio.h` se ejecuta como parte del programa de usuario.
2. **Sistema de colas (*spooling*):**
    - Técnica para manejar **dispositivos de E/S dedicados** en multiprogramación (ej.: impresora). Sin ella, un proceso podría **abrir el dispositivo y no soltarlo** por horas.
    - Un **demonio** (único proceso con permiso para usar el archivo especial del dispositivo) y un **directorio de cola** (*spool*): los procesos dejan allí el archivo completo y el demonio los imprime uno a uno.
    - Linux: `/var/spool/cups/` o `/var/spool/lpd/` . Windows: **Print Spooler** y `C:\Windows\System32\spool\PRINTERS\` .
    - También se usa en la **red** (demonio de red que toma archivos de la cola de red y los transmite).

📌 **Ejemplo Práctico / En Linux:**

```
bash
strace -e trace=write ./programa_con_printf      # printf (biblioteca) -> write
(syscall)
lpstat -o ; ls /var/spool/cups                   # cola de impresión y demonio cupsd
systemctl status cups                            # el demonio de spooling
```

## ✅ Checklist de calidad

- ✓
  Se respondieron **las 19 preguntas**, en el orden temático indicado (numeración original respetada).
- ✓
  Cada una tiene **analogía**, **explicación de cátedra** y **ejemplo en Linux**.
- ✓
  Las preguntas de afirmación (9, 13, 17) comienzan con **VERDADERO** / **FALSO**.
- ✓
  Términos clave preservados: *Port-Mapped I/O*, *Memory-Mapped I/O*, DMA, robo de ciclo, ráfaga, sondeo/espera activa, interrupción precisa/imprecisa, *top/bottom half*, doble búfer, spooling, demonio.

## 📚 Bibliografía


- Tanenbaum, A. — *Sistemas Operativos Modernos*, cap. 5 (Entrada/Salida), secciones 5.1 y 5.2.
- Silberschatz, A. — *Fundamentos de Sistemas Operativos*, cap. 13 (Sistemas de E/S), 13.1.
- Apunte de cátedra: `Unidad 5.pdf`.